MANUAL DE INTEGRACIÓN MQTT

Controladora de Accesos **SWATID A2 / A2v3**

Mensajería completa: eventos emitidos y comandos aceptados

Versión del manual	v1.2
Fecha	Septiembre 2026
Firmware de referencia	v5.0.5 (base, -4G, -BLE, -S3)
Audiencia	Integradores de plataforma / backend

Smart World And Things SLU · CIF B40599813
Tel. +34 633 448 427 · info@swat-id.com · www.swat-id.com

Índice

1. Convenciones generales 3

2. Mapa de topics 3

El equipo PUBLICA en . . . 3

El equipo SE SUSCRIBE a . . . 4

3. Mensajes que EMITE el dispositivo 4

3.1 Keepalive — swatidhome/keepalive/ (cada 60 s) . . . 4

3.2 Solicitud de validación remota — swatidhome/command//access . . . 4

3.3 Evento de acceso — swatidhome/events//access . . . 4

3.4 Acceso fallido — swatidhome/events//failed_access . . . 5

3.5 Evento de torno — swatidhome/events//turnstile . . . 5

3.6 Estado de relés — swatidhome//relay_enable . . . 5

3.7 Entrada digital tipo pulsador — swatidhome//digital_input . . . 5

3.8 Entrada digital tipo puerta — swatidhome//digital_input (v5.0.2) . . . 5

3.9 Información del dispositivo — swatidhome//info . . . 5

3.10 Respuesta a comandos — swatidhome/response//rx . . . 5

3.11 Errores y avisos — swatidhome/errors//rx . . . 6

3.12 Eventos de códigos — swatidhome/events//codes . . . 6

3.13 Eventos BLE — swatidhome/events//ble (solo variante -BLE) . . . 6

3.14 Sincronización de hora — swatidhome//events . . . 6

4. Comandos que RECIBE el dispositivo 6

4.0 Tipo 0 — Activación de relé . . . 6

4.1 Tipo 1 — Solicitud de información . . . 6

4.2 Tipo 2 — Comandos de sistema (message_info es un string) . . . 7

4.3 Tipo 3 — Seguridad . . . 7

4.4 Tipo 4 — Sincronización de hora . . . 7

4.5 Tipo 5 — Gestión de códigos (contrato v5.0.4, validado en placa) . . . 7

4.6 Tipo 6 — Gestión BLE (solo variante -BLE) — LEGADO desde v5.0.5 . . . 8

4.7 Tipo 7 — Gestión de red (WiFi / 4G / RTC) . . . 9

4.8 Canal de vinculación de usuarios (/vinculo, v5.0.5) . . . 9

5. Respuesta de validación remota 10

Formato recomendado (response) . . . 10

Formato legado (access_granted) — solo compatibilidad . . . 11

6. Buenas prácticas de integración 11

7. Compatibilidad y versionado 11

Versión del manual: v1.2 · **Fecha:** Septiembre 2026 **Firmware de referencia:** v5.0.5 (variantes: base, -4G, -BLE, -S3) **Empresa:** Smart World And Things SLU · info@swat-id.com · www.swat-id.com

Documento de referencia para integradores: **toda** la mensajería MQTT del equipo — lo que publica el dispositivo y los comandos que acepta. Extraído y verificado contra el firmware v5.0.5 (gestión de códigos validada en placa el 29-Sep-2026; canal de vinculación de usuarios validado el 01-Oct-2026).

1. Convenciones generales

Concepto	Valor
Identificador del equipo `<id>`	Serial fijo derivado de la MAC, p. ej. `SWATID_A0461CA0ED30`. Inmutable.
Prefijo de topics	`swatidhome/`
Suscripción del equipo	`swatidhome/command/<id>/#`
QoS	0 (publicación simple), sin retain
Formato	JSON UTF-8
`timestamp`	ISO-8601 local, p. ej. `2026-09-22T10:15:00+01:00`
`message_id`	Contador del emisor; en respuestas, eco del id del comando
Buffer MQTT del equipo	4096 bytes desde v5.0.4 (payloads mayores se descartan; los listados se paginan)

Envelope de comando (todo lo que la plataforma envía al equipo):

```
{
  "timestamp": "2026-09-22T10:15:00+01:00",
  "message_id": 123,
  "device": "SWATID_A0461CA0ED30",
  "message_type": 0,
  "message_info": { }
}
```

- `device` debe ser el serial fijo (o el nombre editable del equipo); si no coincide, el comando se ignora.
- `message_type` selecciona la familia de comando (sección 4).
- `message_info` lleva los parámetros (objeto o string según el tipo).

2. Mapa de topics

El equipo PUBLICA en

Topic	Contenido	Sección
`swatidhome/keepalive/<id>`	Latido cada 60 s	3.1
`swatidhome/command/<id>/access`	Solicitud de validación remota (¡el equipo publica en su propio árbol command!)	3.2
`swatidhome/events/<id>/access`	Accesos concedidos	3.3
`swatidhome/events/<id>/failed_access`	Accesos fallidos	3.4
`swatidhome/events/<id>/turnstile`	Eventos del modo torno	3.5
`swatidhome/<id>/relay_enable`	Cambios de estado de relés	3.6
`swatidhome/<id>/digital_input`	Entradas digitales (trigger y door_contact)	3.7, 3.8
`swatidhome/<id>/info`	Información completa del equipo	3.9

Topic	Contenido	Sección
`swatidhome/response/<id>/rx`	Respuestas a comandos	3.10
`swatidhome/errors/<id>/rx`	Errores y avisos	3.11
`swatidhome/events/<id>/codes`	Eventos de gestión de códigos	3.12
`swatidhome/events/<id>/ble`	Eventos BLE (solo variante -BLE)	3.13
`swatidhome/<id>/events`	Evento TIME_SYNC	3.14
`swatidhome/<id>/vinculo`	Respuestas del canal de vinculación (v5.0.5)	4.8

El equipo SE SUSCRIBE a

Topic	Uso
`swatidhome/command/<id>/#`	Todos los comandos (sección 4) y las respuestas de validación (sección 5). La plataforma puede usar cualquier subtopic; se recomienda `.../cmd` para comandos, `.../granted` para respuestas de validación y `.../vinculo` para la gestión de usuarios (4.8).

3. Mensajes que EMITE el dispositivo

3.1 Keepalive — `swatidhome/keepalive/<id>` (cada 60 s)

```
{ "status": "online", "uptime": 1234567 }
```

`uptime` en milisegundos desde el arranque. Recomendación: marcar el equipo offline si no llegan 2-3 latidos consecutivos.

3.2 Solicitud de validación remota — `swatidhome/command/<id>/access`

Emitida cuando un código presentado no se resuelve localmente (o el modo de validación es «remoto primero»). La plataforma debe responder en **< 5 s** (sección 5).

Modo normal:

```
{
  "timestamp": "…", "message_id": 456,
  "device": "SWATID_A0461CA0ED30", "device_name": "Puerta Norte",
  "message_type": 0,
  "message_info": {
    "source": "AUTO",
    "code_type": "PIN",          // "PIN" | "TAG"
    "code_value": "1234",
    "keyboard_id": 1,            // 1 | 2
    "keyboard_name": "WIEGAND1",
    "keyboard_pins": "5/6",
    "request_relay": 1,
    "max_duration": 2.0
  }
}
```

Modo torno: añade `"mode": "turnstile"`, `message_info.actual_relay` (relé que abrirá realmente) y el objeto `turnstile_info` (`enabled`, `keyboard1_relay`, `keyboard2_relay`, `timeout`, `pending_request`).

3.3 Evento de acceso — `swatidhome/events/<id>/access`

```
{
  "timestamp": "…", "message_id": 457,
  "device": "Puerta Norte", "serial": "SWATID_A0461CA0ED30",
  "event_type": "ACCESS_EVENT",
  "success": true,
  "source": "LOCAL",             // LOCAL | REMOTE | REMOTE_APPROVED | WEB | MQTT_RELAY | REMOTE_COMMAND
  "code_type": "PIN", "code_value": "1234",
```

```
  "keyboard_id": 1, "keyboard_name": "WIEGAND1"   // si aplica
}
```

3.4 Acceso fallido — `swatidhome/events/<id>/failed_access`

Como 3.3 con `event_type: "FAILED_ACCESS"` y además: `reason` (`INVALID_CODE` | `BLOCKED` | `REMOTE_DENIED: <detalle>` …), `failed_attempts`, `max_attempts`.

3.5 Evento de torno — `swatidhome/events/<id>/turnstile`

```
{
  "timestamp": "…", "message_id": 458,
  "device": "SWATID_A0461CA0ED30", "device_name": "Puerta Norte",
  "message_type": 1, "mode": "turnstile",
  "event_type": "ACCESS_GRANTED",      // ACCESS_GRANTED | ACCESS_DENIED
  "event_info": {
    "code_type": "PIN", "code_value": "1234",
    "keyboard_id": 1, "keyboard_name": "WIEGAND1",
    "success": true,
    "reason": "TORNO_LOCAL",           // TORNO_LOCAL | TORNO_REMOTE | TORNO_REMOTE_LOCAL | TORNO_LOCAL_FALLBACK_MQTT_
    "relay_opened": 1, "duration": 2.0
  },
  "turnstile_info": { "enabled": true, "keyboard1_relay": 1, "keyboard2_relay": 2, "timeout": 5000 }
}
```

3.6 Estado de relés — `swatidhome/<id>/relay_enable`

En cada activación y en el apagado automático:

```
{ "timestamp": "…", "resultado": "OK", "rele": 1,
  "estado": "ON",                 // ON | OFF
  "origen": "SYSTEM",             // SYSTEM (activación) | TIMEOUT (apagado)
  "message_id": "88", "request_id": "12" }
```

3.7 Entrada digital tipo pulsador — `swatidhome/<id>/digital_input`

`event: "digital_input_trigger"` (contrato estable desde v3.x):

```
{ "timestamp": "…", "event": "digital_input_trigger",
  "input": 1, "gpio": 16, "relay": 1,
  "duration_ms": 2000, "duration": 2.0, "message_id": "89" }
```

Casos especiales del modo Inverso: activación mantenida `duration_ms: 4294967295`; desactivación `duration_ms: 0`.

3.8 Entrada digital tipo puerta — `swatidhome/<id>/digital_input` (v5.0.2)

`event: "door_contact"` — **mismo topic, evento distinto**; nunca incluye `relay` ni `duration`:

```
{ "timestamp": "…", "event": "door_contact",
  "input": 2, "gpio": 17,
  "door_state": "open",          // open | closed
  "raw_level": "HIGH", "open_level": "HIGH",
  "reason": "change",            // boot | change | sync
  "message_id": "90" }
```

`reason=boot/sync` es un estado absoluto (no una transición): `boot` al arrancar; `sync` tras recuperar el broker (pudieron perderse cambios).

3.9 Información del dispositivo — `swatidhome/<id>/info`

Respuesta al comando tipo 1 (también se copia en `response/<id>/rx`). Campos principales: `device`, `serial`, `ip`, `mac`, `firmware_version`, `use_dhcp`, `relay_duration`; objeto `security` (bloqueos, intentos); objeto `keyboards` (pines Wiegand); array `stored_codes` (códigos locales completos); objeto `local_codes_info` (`count`, `max_capacity`, `available`); array `stored_remote_codes` (con `time_slots` y `days_string`); objeto `remote_codes_info`. Con IP estática añade `static_ip/gateway/subnet/dns`.

3.10 Respuesta a comandos — `swatidhome/response/<id>/rx`

Toda ejecución de comando produce una respuesta:

```
{ "timestamp": "…", "response_id": 31, "message_id": 123,
  "device": "Puerta Norte", "serial": "SWATID_A0461CA0ED30",
  "response_type": 0,             // 0 = OK, 1 = error
  "response_info": "relay 1 activated",
  "relay": 1,                     // solo en comandos de relé
  "data": { }                     // v5.0.4: objeto estructurado (gestión de códigos)
}
```

`message_id` es el eco del comando original — usarlo para correlar. `response_info` puede ser texto o un JSON serializado (get_wifi, get_gsm, list_local_codes, get_status BLE…).

Desde v5.0.4, las respuestas de la gestión de códigos (tipo 5) añaden el objeto `data` con el resultado estructurado (ver 4.5) — el campo `response_info` se mantiene por compatibilidad. Recomendación para integraciones nuevas: parsear `data` y usar `response_info` solo como texto de log.

3.11 Errores y avisos — `swatidhome/errors/<id>/rx`

```
{ "timestamp": "…", "message_id": 91, "device": "Puerta Norte",
  "serial": "SWATID_A0461CA0ED30", "error_code": 3,
  "description": "Dispositivo iniciado - Dual Wiegand v5.0.2" }
```

error_code	Significado
3	Arranque del equipo (informativo)
4	Cambio de seguridad desde web (bloqueos)
5	Bloqueo temporal levantado automáticamente
7	Mensaje de validación remota inválido
8	Memoria baja
9	(reservado histórico)

3.12 Eventos de códigos — `swatidhome/events/<id>/codes`

Alta de código desde BLE (variante -BLE): `event: "CODE_ADDED"` con `type`, `value`, `keyboard_id`, `relay`, `source: "BLE"`, `timestamp`.

3.13 Eventos BLE — `swatidhome/events/<id>/ble` (solo variante -BLE)

`event ∈ AUTH_SUCCESS, AUTH_FAILED, AUTH_TIMEOUT, RELAY_ACTIVATED, RELAY_DEACTIVATED, USER_ADDED, USER_ADDED_DERIVED, SUPERADMIN_SET, BINDINGS_CLEARED`. Campos habituales: `device`, `user`/`slot`/`name` según evento, `source` (`BLE`|`MQTT`|`WEB`), `timestamp`.

3.14 Sincronización de hora — `swatidhome/<id>/events`

Tras aceptar un comando tipo 4: `event_type: "TIME_SYNC"` con `time_string` y `source: "MQTT"`.

4. Comandos que RECIBE el dispositivo

Publicar en `swatidhome/command/<id>/cmd` (cualquier subtopic sirve) con el envelope de la sección 1. Todas las familias responden por `response/<id>/rx` (3.10).

4.0 Tipo 0 — Activación de relé

```
"message_type": 0,
"message_info": { "relay_number": 1, "duration": 3.5 }
```

`relay_number` 1|2; `duration` ≥ 0.5 s. Respuesta: `"relay 1 activated"`. Genera además evento de acceso (`source: REMOTE_COMMAND`) y mensajes `relay_enable`.

4.1 Tipo 1 — Solicitud de información

```
"message_type": 1
```

Publica el estado completo en `<id>/info` (3.9) y lo copia en la respuesta.

4.2 Tipo 2 — Comandos de sistema (`message_info` es un string)

`message_info`	**Efecto**
`"reboot"`	Reinicio del equipo
`"reset"`	Restaura configuración de fábrica
`"update"`	Aplica configuración: el mismo mensaje puede llevar campos extra en un objeto `message_info` con `device_name`, `relay_duration`, `use_dhcp`, `static_ip/gateway/subnet/dns`
`"ota_config"`	Configura OTA (`update_url`, `check_interval` en un objeto)
`"ota_check"`	Fuerza comprobación de actualización

4.3 Tipo 3 — Seguridad

`message_info.security_command`:

Comando	**Parámetros**	**Efecto**
`block_local_access`	—	Bloquea validación local
`unblock_local_access`	—	Desbloquea y resetea intentos
`disable_keyboard_reading`	—	Ignora los teclados
`enable_keyboard_reading`	—	Reactiva los teclados
`set_block_duration`	`duration_seconds`	Duración del bloqueo automático
`set_max_failed_attempts`	`max_attempts`	Umbral de intentos fallidos

4.4 Tipo 4 — Sincronización de hora

```
 "message_type": 4,
 "message_info": { "time_string": "2026-09-22 10:15:00" }
```

Actualiza la hora del equipo (en A2v3 también reloj del sistema + RTC DS3231). Emite el evento TIME_SYNC (3.14).

4.5 Tipo 5 — Gestión de códigos (contrato v5.0.4, validado en placa)

`message_info.action`. Todas las respuestas incluyen el objeto `data` (3.10) además del texto legacy. Validaciones comunes: `code_type` ∈ {`PIN`, `TAG`}; `code_value` 1–16 caracteres; `keyboard_id` 0 (ambos) | 1 | 2; `relay` 1 | 2. Cualquier parámetro inválido responde `response_type: 1` con el motivo exacto en `response_info`.

Semántica: los códigos **locales** son siempre activos (sin calendario); los **remotos** (caché NVS) admiten hasta 4 franjas horarias — sin `time_slots` o con `[]` quedan **siempre activos** (`always_active: true`).

Acción	**Parámetros**	**Respuesta** `data`
`list_local_codes`	`offset` (def. 0), `limit` (def. 50, máx. 50)	`scope`, `count`, `max`, `offset`, `limit`, `validation_mode`, `always_active`, `codes[]`, `showing`, `truncated`, `next_offset`
`add_local_code`	`code_type`, `code_value`, `relay`, `keyboard_id` opc. (def. 0). `time_slots` → **error**	eco + `always_active: true`, `total_codes`
`remove_local_code`	`code_type`, `code_value`, `keyboard_id` opc. (filtro)	eco + `remaining`
`clear_local_codes`	—	`cleared_count`
`list_remote_codes` **(nueva)**	`offset` (def. 0), `limit` (def. 8, máx. 10)	como el local + por código: `always_active`, `time_slots_count`, `time_slots[]` (con `days_string`)

Acción	Parámetros	Respuesta `data`
`add_remote_code`	`code_type`, `code_value`, `keyboard_id`, `relay`, `time_slots[]` opc. (máx. 4, cada uno COMPLETO y válido)	eco + `always_active`, `time_slots_count`, `total_codes`
`remove_remote_code`	`code_type`, `code_value`	eco + `remaining`
`clear_remote_codes`	—	`cleared_count`

Franja horaria (todos los campos obligatorios; una franja incompleta o inválida **rechaza el alta entera**):

```
{ "start_hour": 8, "start_minute": 0, "end_hour": 20, "end_minute": 0,
  "days_of_week": 31 }
```

`days_of_week`: bitmask 1=Lun, 2=Mar, 4=Mié, 8=Jue, 16=Vie, 32=Sáb, 64=Dom (rango válido 1–127; 31=Lun-Vie, 127=todos).

Ejemplo — `list_remote_codes` (captura real):

```
"data": {
  "scope": "remote", "count": 2, "max": 40, "offset": 0, "limit": 8,
  "codes": [
    { "id": 0, "type": "PIN", "value": "5555", "keyboard_id": 0, "relay": 1,
      "always_active": true, "time_slots_count": 0, "time_slots": [] },
    { "id": 1, "type": "PIN", "value": "6666", "keyboard_id": 0, "relay": 2,
      "always_active": false, "time_slots_count": 1,
      "time_slots": [ { "start_hour": 8, "start_minute": 0, "end_hour": 20,
        "end_minute": 0, "days_of_week": 31, "days_string": "Lun-Vie" } ] }
  ],
  "showing": 2, "truncated": false
}
```

Paginación: si `truncated: true`, pedir la página siguiente con `offset = next_offset`. En `list_local_codes` el string de `response_info` duplica el JSON solo hasta 1,5 KB (en páginas mayores lleva un resumen); `codes[]` local incluye `keyboard_id` y el alias legado `keyboard`.

Errores típicos (`response_type: 1`): `local codes do not support time_slots (use add_remote_code)` · `time_slots[i] incomplete (…)` · `time_slots[i] invalid (hours 0-23, minutes 0-59, days_of_week 1-127)` · `too many time_slots (max 4)` · `invalid relay (1 or 2)` · `local/remote code not found` · `unknown action: <acción>`.

4.6 Tipo 6 — Gestión BLE (solo variante -BLE) — LEGADO desde v5.0.5

> ■■ **Deprecado para integraciones nuevas.** La gestión de usuarios se hace > por el canal `/vinculo` (sección 4.8), con autenticación HMAC propia. El > tipo 6 se mantiene por compatibilidad (su única autenticación son las > credenciales del broker) y no se ampliará.

`message_info.action`:

Acción	Parámetros	Notas
`set_superadmin`	`key` (hex 128 chars)	Clave del superadministrador
`add_user`	`slot` 1-5, `key` (hex 128), `name`, `permissions`	Permisos: bitmask 1=relés, 2=modo, 4=códigos, 8=red, 255=admin
`add_user_derived`	`slot`, `user_id`, `name`, `permissions`	Clave derivada por HKDF; la respuesta incluye `salt` e `info_prefix` para que la app derive la misma clave
`clear_user`	`slot`	
`clear_superadmin`	—	
`clear_all`	—	Borra todas las vinculaciones
`get_status`	—	Respuesta: superadmin, usuarios activos, máx.
`list_users`	—	Detalle de usuarios (con vista previa de claves, nunca completas)

4.7 Tipo 7 — Gestión de red (WiFi / 4G / RTC)

`message_info.action`:

Acción	Parámetros	Respuesta
`get_wifi`	—	JSON de estado: `ap{active,mode,timeout_s,remaining_s,clients,ssid,ip,pass_default}`, `sta{enabled,ssid,connected,ip,rssi}`, `eth_up`, `eth_ip`, `gsm_up`, `mqtt_iface`, `mqtt_connected`
`set_wifi`	`ap_mode` (0=off,1=ventana,2=siempre), `ap_timeout_s`, `ap_pass` (≥8), `sta_ssid+sta_pass`, `sta_enabled`, `forget_sta` — todos opcionales	Resumen de lo aplicado
`ap_start`	`seconds` (0=ventana configurada)	Activa el AP ya
`ap_stop`	—	Apaga el AP
`wifi_scan`	—	Lanza escaneo async
`get_wifi_scan`	—	`{scanning, networks[{ssid,rssi,enc}]}`
`get_gsm`	—	JSON: `state` (probing/absent/sim_check/pin_required/pin_error/puk_required/no_sim/registering/attached), `model`, `imei`, `operator`, `csq` (99=sin señal), `rssi_dbm`, `pin_set`, `pin_attempts` (PIN1,PIN2,PUK1,PUK2), `reg_stat+reg_stat_text`, `radio` (CPSI), `data_started`, `data_up`, `data_ip`, `apn_active`, `apn_mode`, `apn`, `last_error`
`set_gsm`	`enabled`, `pin` (solo escritura; UN intento por valor), `pin_disable` (desbloquea y quita el PIN de la SIM permanentemente), `apn+apn_mode+apn_user+apn_pass`	Resumen
`gsm_rescan`	—	Reinicia el módulo 4G (AT+CRESET) y re-detecta SIM
`get_rtc`	—	`{present, osf, seeded}` (solo A2v3)

4.8 Canal de vinculación de usuarios (`/vinculo`, v5.0.5)

Gestión uniforme de usuarios alineada con el resto de productos SWATID (X1). Disponible en TODAS las variantes (con y sin BLE).

Dirección	Topic
Backend → equipo	`swatidhome/command/<id>/vinculo`
Equipo → backend	`swatidhome/<id>/vinculo`
Eventos `BINDING_ADDED/UPDATED/CLEARED`	`swatidhome/events/<id>/ble`

Roles: *superadmin de plataforma* (clave de servicio SWATID — siempre accede; solo se gestiona por procedimiento de fábrica en la web del equipo), *admin de placa* (slot 0, alta por TOFU) y *usuarios* (slots 1–5, permisos bitmask `0x01` relé · `0x02` modo · `0x04` códigos · `0x08` red).

Comandos públicos (payload JSON plano):

```
{"command":"devinfo","type":"GET"}
{"command":"challenge","type":"GET"}
{"command":"link","type":"POST","key":"<128 hex>"}
```

- `devinfo` → `capabilities{board,relays,keyboards,digital_inputs,slots,ble,gsm,mqtt}`, `superadmin_registered` (admin de placa, alias X1), `admin_registered`, `platform_superadmin_registered`, `available_for_linking`, `slots_used`, `slots_available`, `remote_link_window_open`, `remote_link_window_s`.

- `challenge` → `nonce` 64 hex, `expires_s: 60`. Un solo nonce vivo; se **consume** al primer intento de uso (anti-replay).
- `link` público = TOFU del admin de placa: solo sin admin previo y con la **ventana remota abierta** (10 min tras arrancar, o abierta desde la web `/binding`). Éxito: `{"linked":true,"slot":0,"role":"admin","role_alias":"superadmin"}`.

Sobre autenticado (resto de comandos):

```
{"nonce":"<64 hex>","hmac":"<64 hex>","payload":"<comando JSON como string>"}
```

`hmac = HMAC-SHA256(clave, unhex(nonce) ■ utf8(payload))`. El equipo prueba plataforma → admin → usuarios 1..5; el primero que verifica es el firmante.

Comando (en `payload`)	Quién	Efecto
`{"command":"link","type":"POST","slot":N,"new_key":"<128hex>","name":"…","permissions":P}`	admin / plataforma	Alta o actualización del usuario N (1–5)
`{"command":"update_vinculo","new_key":"<128hex>"}`	cualquiera	Rota la clave del firmante
`{"command":"unlink"}`	usuario	Baja propia
`{"command":"unlink"}`	admin	Baja del admin **y de todos los usuarios** (la plataforma queda intacta)
`{"command":"unlink","slot":N}`	admin / plataforma	Baja del usuario N
`{"command":"unlink","slot":0}`	solo plataforma	Quita al admin (y usuarios)
`{"command":"list"}`	admin / plataforma	Vínculos (slot, role, name, permissions — nunca claves)

Errores: `missing_key`, `invalid_key`, `invalid_json`, `auth_required`, `invalid_signature`, `nonce_expired`, `device_already_owned`, `remote_linking_window_closed`, `invalid_new_key_or_slot`, `admin_required`, `platform_required`, `factory_procedure_required`, `not_linked`, `nvs_write_failed`, `unknown_command`.

Firma de ejemplo (Python):

```
import hashlib, hmac, json
payload = json.dumps({"command": "link", "type": "POST", "slot": 2,
                      "new_key": NEW_KEY, "name": "Juan", "permissions": 3},
                     separators=(",", ":"))
mac = hmac.new(bytes.fromhex(KEY_ADMIN),
               bytes.fromhex(nonce) + payload.encode(),
               hashlib.sha256).hexdigest()
envelope = {"nonce": nonce, "hmac": mac, "payload": payload}
```

En builds **-BLE**, las mismas claves (plataforma, admin, usuarios) sirven para el challenge-response BLE (FF0B/FF01); la clave «superadmin» BLE histórica se migra automáticamente como admin de placa.

5. Respuesta de validación remota

La plataforma responde a la solicitud 3.2 publicando en `swatidhome/command/<id>/granted` (o cualquier subtopic de command). Dos formatos aceptados:

Formato recomendado (`response`)

```
{
  "device": "SWATID_A0461CA0ED30",
  "message_id": 456,               // eco del message_id de la solicitud
  "response": "APPROVED",          // APPROVED | DENIED
  "code_type": "PIN", "code_value": "1234",
  "relay_number": 1,               // relé a abrir (modo normal)
  "duration": 3.0,                 // segundos (opcional)
  "reason": "Usuario autorizado"
}
```

• **Timeout: 5 s.** Sin respuesta, en modo torno se registra TIMEOUT (y el equipo aplicó fallback local si el código existía localmente).

• En **modo torno** el relé lo decide el equipo (`actual_relay` de la solicitud); `message_id` debe coincidir con el de la solicitud pendiente.

Formato legado (`access_granted`) — solo compatibilidad

```
{ "access_granted": true, "code_type": "PIN", "code_value": "1234",
  "relay_number": 1, "duration": 3000, "reason": "…" }
```

Nota: aquí `duration` va en **milisegundos**. No usar en integraciones nuevas.

6. Buenas prácticas de integración

• **Filtrar siempre por** `event`/`event_type` — un mismo topic puede transportar varios contratos (p. ej. `digital_input`: trigger y door_contact). Ignorar eventos desconocidos sin error.

• **Tolerar campos nuevos**: las ampliaciones añaden claves, nunca cambian las existentes.

• **Correlación**: usar `message_id` (comando) ↔ `message_id` (respuesta).

• **Validación remota**: responder en < 5 s; tratar `reason=boot|sync` de door_contact como estado absoluto.

• **Presencia**: keepalive 60 s + LWT no configurado — usar ausencia de keepalives.

• **No asumir orden** entre evento de acceso y relay_enable.

• El equipo funciona sin broker (validación local); al reconectar sincroniza estados de puerta (`reason: sync`) pero **no** reenvía eventos de acceso perdidos.

7. Compatibilidad y versionado

Versión firmware	Cambios de mensajería
v3.x	Base: validación, eventos, relés, `digital_input_trigger`, tipos 0-5
v4.x (-BLE)	Tipo 6 BLE + eventos BLE
v5.0.0/v5.0.1	Tipo 7 red (WiFi/AP/4G/RTC); `get_gsm` ampliado (reg_stat, radio, data_*, pin_attempts)
v5.0.2	`door_contact` (nuevo evento, mismo topic); campos `diN_type`/`open_level` en APIs; `set_gsm.pin_disable`
v5.0.4	Tipo 5 completado: `list_remote_codes`, paginación `offset`/`limit`, objeto `data` en respuestas, validación estricta de altas (franjas incluidas), rechazo de `time_slots` en locales; buffer MQTT 4 KB
v5.0.5	Canal `/vinculo` (gestión de usuarios con HMAC, 3 roles, ventana TOFU); eventos `BINDING_*`; tipo 6 queda LEGADO; web `/binding` (fábrica)

Regla general: los contratos existentes no se modifican; toda ampliación es un `event` nuevo o campos añadidos.

Manual de Integración MQTT v1.2 — Octubre 2026 — Smart World And Things SLU.